# Colecții și genericitate

*Seminar 3 – Set, Map, contractul equals/hashCode, Comparator, InMemoryRepository*

## Cerința 1. Clasa Student

> Creați o clasă `Student` având următorii membri: `nume` (`String`), `media` (`float`), un constructor cu parametri care inițializează un student și metoda `toString()`.

Instanțiați următorii studenți:

```
Student s1 = new Student("Dan", 4.5f);
Student s2 = new Student("Ana", 8.5f);
Student s3 = new Student("Dan", 4.5f);
```

## Cerința 2. HashSet

> Instanțiați un obiect de tip `HashSet<Student>` și adăugați studenții de la cerința 1.

### Întrebare

Ce observați?

## Cerința 3. Contractul equals – hashCode

> Respectați contractul **equals – hashCode**: dacă `obj1.equals(obj2)`, atunci `obj1.hashCode() == obj2.hashCode()`.

### Observație

Atunci când doriți să salvați obiecte în colecții reprezentate în memorie prin tabele de dispersie (hash table), dacă implementați `equals()` trebuie să implementați și `hashCode()`.

## Cerința 4. TreeSet și Comparator

> Instanțiați un obiect de tipul `TreeSet<Student>` și adăugați studenții de la cerința 1. Definiți un comparator care compară doi studenți după nume.

## Cerința 5. HashMap și TreeMap

> Repetați cerințele 2–4 folosind `HashMap` și `TreeMap`.

## Cerința 6. Clasa MyMap

> Scrieți o clasă `MyMap`, care va reprezenta un `Map` pentru reținerea studenților după medie.

Caracteristicile clasei:

- Cheile pot avea valori de la 0 la 10 (corespunzătoare mediilor posibile).
- Valoarea asociată fiecărei chei va fi o listă (`List`) care va reține toți studenții cu media rotunjită egală cu cheia. Considerăm că un student are media rotunjită 8 dacă media sa este în intervalul [7.50, 8.49] (`Math.round`).

- Map-ul va menține cheile (mediile) ordonate descrescător. Folosiți o implementare potrivită a interfeței `Map`, care să permită acest lucru, și definiți un `Comparator` pentru stabilirea ordinii cheilor (clasă internă statică).

### Pași

1. Definiți în clasa `MyMap` metoda `add(Student)`, care va adăuga un student în lista corespunzătoare mediei lui. Dacă nu există deja niciun student cu media respectivă (rotunjită), lista va fi creată la cerere.
2. Adăugați câțiva studenți.
3. Definiți o metodă `getEntries()` care returnează mulțimea intrărilor – `Entry<Key, Value>`.
4. Creați un dicționar de tipul `MyMap` și adăugați următorii studenți:

```
public static List<Student> getList() {
    List<Student> l = new ArrayList<Student>();
    l.add(new Student("1", 9.7f));
    l.add(new Student("2", 7.3f));
    l.add(new Student("3", 6f));
    l.add(new Student("4", 6.9f));
    l.add(new Student("5", 9.5f));
    l.add(new Student("6", 9.9f));
    return l;
}
```

5. Iterați mulțimea intrărilor – `Entry<Key, Value>` – și sortați alfabetic fiecare listă de studenți.

### Discuție

Ce se întâmplă când apelăm metoda `put` moștenită de la dicționar? **„Favor Composition over Inheritance”**

## Cerința 7. Interfața Repository

> Considerăm interfața `Repository0`, care specifică operațiile CRUD pentru un repository cu elemente generice, care au un id de tip generic `ID`.

```
/**
 * CRUD operations repository interface
 * @param <ID> - type E must have an attribute of type ID
 * @param <E> - type of entities saved in repository
 */
public interface Repository0<ID, E extends Entity<ID>> {
    /**
     * @param id - the id of the entity to be returned
     *           id must not be null
     * @return the entity with the specified id
     *         or null - if there is no entity with the given id
     * @throws IllegalArgumentException
     *         if id is null.
     */
    E findOne(ID id);

    /**
     * @return all entities
     */
    Iterable<E> findAll();

    /**
     * @param entity
     *        entity must be not null
     * @return null - if the given entity is saved
     *         otherwise returns the entity (id already exists)
```

```
     * @throws ValidationException
     *         if the entity is not valid
     * @throws IllegalArgumentException
     *         if the given entity is null.
     */
    E save(E entity);

    /**
     * removes the entity with the specified id
     * @param id
     *        id must be not null
     * @return the removed entity or null if there is no entity with the given id
     * @throws IllegalArgumentException
     *         if the given id is null.
     */
    E delete(ID id);

    /**
     * @param entity
     *        entity must not be null
     * @return null - if the entity is updated,
     *         otherwise returns the entity - (e.g. id does not exist).
     * @throws IllegalArgumentException
     *         if the given entity is null.
     * @throws ValidationException
     *         if the entity is not valid.
     */
    E update(E entity);
}
```

## Cerința 8. InMemoryRepository

> Implementați o clasă `InMemoryRepository` care să implementeze interfața `Repository0`. Creați o instanță de `InMemoryRepository` și adăugați studenții de la cerința 1.